

4.4 랜덤 포레스트(Random Forest)

목차

1. 랜덤 포레스트 개요
 1. 배깅(Bagging)과 랜덤 포레스트
 2. 부트스트래핑 샘플링
 3. 장단점
2. 중복 피쳐명 처리 함수
3. 랜덤 포레스트 기본 성능
4. GridSearchCV로 하이퍼 파라미터 튜닝
 1. 탐색 파라미터 설정
 2. 탐색 결과 분석
5. 최적 파라미터로 재학습
6. 피쳐 중요도 시각화
7. 핵심 요약 및 머신러닝 활용 포인트

1. 랜덤 포레스트 개요

1.1 배깅(Bagging)과 랜덤 포레스트

배깅(Bagging)은 4.3장에서 배운 앙상블 유형 중 하나로, **Bootstrap Aggregating**의 줄임말입니다. 보팅이 서로 다른 알고리즘을 결합하는 반면, 배깅은 **같은 알고리즘으로 여러 개의 분류기를 만들어 보팅으로 최종 결정**합니다.

랜덤 포레스트(Random Forest)는 배깅의 대표적인 알고리즘입니다. 이름 그대로 **여러 개의 결정 트리(Tree)가 모여 숲(Forest)**을 이룬 형태입니다.

특징	설명
기반 알고리즘	전부 결정 트리 . 결정 트리의 쉽고 직관적인 장점 을 그대로 가집니다.
데이터 샘플링	각 트리가 부트스트래핑 으로 서로 다르게 샘플링된 데이터를 학습합니다.
피쳐 샘플링	분할 시 전체 피쳐 중 일부만 무작위로 후보에 넣습니다. 이것이 '랜덤'의 또 다른 의미입니다.
최종 결정	모든 트리의 예측을 소프트 보팅 으로 결합합니다.

■ 왜 랜덤성을 두 번 주는가

데이터만 다르게 샘플링하면 트리들이 **여전히 비슷하게** 만들어집니다. 강한 피쳐 하나가 있으면 모든 트리가 그 피쳐로 먼저 분할하기 때문입니다.

그래서 랜덤 포레스트는 **분할할 때마다 피쳐도 무작위로 일부만** 후보에 넣습니다. 이렇게 하면 트리들이 **서로 충분히 달라져** 앙상블 효과가 극대화됩니다. 4.3장에서 강조한 **"다양성이 핵심"**이라는 원칙의 구체적 구현입니다.

1.2 부트스트래핑 샘플링

부트스트래핑(Bootstrapping)은 원본 데이터에서 **중복을 허용하여** 원본과 같은 크기의 샘플을 여러 개 만드는 방식입니다.

■ 부트스트래핑의 성질

- 중복을 허용하므로 **어떤 데이터는 여러 번 뽑히고**, 어떤 데이터는 **한 번도 뽑히지 않습니다**.
- 수학적으로 각 샘플에서 **약 63.2%의 고유 데이터**만 포함되고 나머지 36.8%는 제외됩니다.
- 제외된 데이터를 **OOB(Out Of Bag)**라고 하며, 별도의 검증 데이터처럼 활용할 수 있습니다(`oob_score=True`).

1.3 장단점

장점	단점
과적합에 강합니다. 여러 트리의 평균을 내므로 개별 트리의 과적합이 상쇄됩니다.	하이퍼 파라미터가 많고 튜닝 시간이 오래 걸립니다. 트리 개수만큼 학습해야 하기 때문입니다. 또한 트리 하나 하나는 해석 가능하지만 전체 숲을 해석하기는 어렵습니다 .
빠른 수행 속도. 트리들이 서로 독립적이라 병렬 처리 가 가능합니다(<code>n_jobs=-1</code>).	
다양한 영역에서 안정적인 성능을 내며, 스케일링이 필요 없습니다.	

2. 중복 피처명 처리 함수

4.2장과 동일한 HAR 데이터를 사용하므로, **중복 피처명을 고유하게 만드는 함수**를 먼저 정의합니다.

```
def get_new_feature_name_df(old_feature_name_df):
    feature_dup_df = pd.DataFrame(data=old_feature_name_df.groupby('column_name').cumcount(),
                                   columns=['dup_cnt'])
    feature_dup_df = feature_dup_df.reset_index()
    new_feature_name_df = pd.merge(old_feature_name_df.reset_index(), feature_dup_df, how='outer')
    new_feature_name_df['column_name'] = new_feature_name_df[['column_name', 'dup_cnt']].apply(
        lambda x : x[0]+'_'+str(x[1]) if x[1] >0 else x[0] , axis=1)
    new_feature_name_df = new_feature_name_df.drop(['index'], axis=1)
    return new_feature_name_df
```

줄	코드	상세 설명
2~3	groupby('column_name').cumcount()	같은 피처명이 몇 번째로 등장하는지 0부터 시작하는 순번 을 매깁니다. HAR 데이터에는 42개의 중복 피처명 이 있어 이 처리가 필수입니다.
5	pd.merge(..., how='outer')	원본 피처명과 순번을 인덱스 기준으로 병합 합니다.
6~7	apply(lambda x : x[0]+'_'+str(x[1]) if x[1]>0 else x[0], axis=1)	순번이 0보다 클 때만 순번 을 덧붙입니다. 첫 등장은 원래 이름을 유지합니다.
8	drop(['index'], axis=1)	병합용 임시 컬럼을 제거합니다.

```
import pandas as pd

def get_human_dataset( ):
    feature_name_df = pd.read_csv('./human_activity/features.txt', sep='\s+',
                                   header=None, names=['column_index', 'column_name'])
    new_feature_name_df = get_new_feature_name_df(feature_name_df)
    feature_name = new_feature_name_df.iloc[:, 1].values.tolist()
    X_train = pd.read_csv('./human_activity/train/X_train.txt', sep='\s+', names=feature_name )
    X_test = pd.read_csv('./human_activity/test/X_test.txt', sep='\s+', names=feature_name)
    y_train = pd.read_csv('./human_activity/train/y_train.txt', sep='\s+', header=None, names=['action'])
    y_test = pd.read_csv('./human_activity/test/y_test.txt', sep='\s+', header=None, names=['action'])
    return X_train, X_test, y_train, y_test

X_train, X_test, y_train, y_test = get_human_dataset()
```

줄	코드	상세 설명
3	def get_human_dataset():	4.2장에서 만든 함수를 그대로 재사용 합니다. 학습 7,352건 , 테스트 2,947건 , 각각 561개 피처 입니다.
14	X_train, X_test, y_train, y_test = get_human_dataset()	4개 데이터 세트를 한 번에 반환받습니다.

3. 랜덤 포레스트 기본 성능

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import pandas as pd
import warnings
```

```
warnings.filterwarnings('ignore')

# 결정 트리에서 사용한 get_human_dataset( )을 이용해 학습/테스트용 DataFrame 반환
X_train, X_test, y_train, y_test = get_human_dataset()

# 랜덤 포레스트 학습 및 별도의 테스트 셋으로 예측 성능 평가
rf_clf = RandomForestClassifier(random_state=0)
rf_clf.fit(X_train, y_train)
pred = rf_clf.predict(X_test)
accuracy = accuracy_score(y_test, pred)
print('랜덤 포레스트 정확도: {0:.4f}'.format(accuracy))
```

줄	코드	상세 설명
1	from sklearn.ensemble import RandomForestClassifier	앙상블 모듈에서 랜덤 포레스트를 임포트합니다. 회귀는 <code>RandomForestRegressor</code> 입니다.
11	rf_clf = RandomForestClassifier(random_state=0)	모든 하이퍼 파라미터를 기본값으로 생성합니다. 사이킷런 0.22부터 <code>n_estimators</code> 기본값이 100이므로, 결정 트리 100개가 만들어집니다.
12~15	fit / predict / accuracy_score	학습·예측·평가는 결정 트리와 완전히 동일한 인터페이스입니다. 클래스 이름만 바뀌었습니다.

▶ 실행 결과

랜덤 포레스트 정확도: 0.9223
(학습 5.6초)

■ 결과 해석 — 앙상블의 위력

모델	테스트 정확도
결정 트리 (기본)	0.8548
결정 트리 (튜닝 완료)	0.8717
랜덤 포레스트 (기본)	0.9223

아무런 튜닝 없이도 랜덤 포레스트가 정성껏 튜닝한 결정 트리보다 5%p 이상 높습니다. 단일 트리가 과적합으로 놓쳤던 패턴을 100개 트리의 다수결이 보정한 결과입니다.

학습 시간도 5.6초로, 트리 100개를 만들었음에도 병렬 처리 덕분에 빠릅니다.

⚠ 경고 메시지에 대하여

`rf_clf.fit(X_train, y_train)` 에서 `y_train` 이 (7352, 1) 2차원 DataFrame이라 사이킷런이 `DataConversionWarning` 을 발생시킵니다. 코드 상단의 `warnings.filterwarnings('ignore')` 가 이를 숨기고 있을 뿐입니다.

안정화 버전에서는 1차원으로 변환해 전달하는 것이 바람직합니다.

```
rf_clf.fit(X_train, y_train.values.ravel())
```

4. GridSearchCV로 하이퍼 파라미터 튜닝

4.1 탐색 파라미터 설정

```
from sklearn.model_selection import GridSearchCV

params = {
    'n_estimators':[100],
    'max_depth' : [6, 8, 10, 12],
    'min_samples_leaf' : [8, 12, 18 ],
    'min_samples_split' : [8, 16, 20]
}
# RandomForestClassifier 객체 생성 후 GridSearchCV 수행
rf_clf = RandomForestClassifier(random_state=0, n_jobs=-1)
grid_cv = GridSearchCV(rf_clf , param_grid=params , cv=2, n_jobs=-1 )
grid_cv.fit(X_train , y_train)

print('최적 하이퍼 파라미터:\n', grid_cv.best_params_)
print('최고 예측 정확도: {0:.4f}'.format(grid_cv.best_score_))
```

줄	코드	상세 설명
4	'n_estimators':[100]	트리 개수를 100 하나로 고정 합니다. 값이 하나뿐이라 탐색 대상은 아니지만, 명시적으로 기록 하기 위해 넣었습니다. 튜닝 단계에서는 시간 절약을 위해 적게 두는 것이 일반적입니다.
5	'max_depth' : [6, 8, 10, 12]	트리 깊이 4가지 . 결정 트리 실습에서 8~10이 좋았던 경험을 반영한 범위입니다.
6	'min_samples_leaf' : [8, 12, 18]	리프 최소 샘플 수 3가지 . 기본값 1보다 크게 잡아 과적합을 억제합니다.
7	'min_samples_split' : [8, 16, 20]	분할 최소 샘플 수 3가지 . 총 조합은 $1 \times 4 \times 3 \times 3 = 36$ 가지입니다.
10	RandomForestClassifier(random_state=0, n_jobs=-1)	n_jobs=-1 은 모든 CPU 코어를 사용하라는 의미입니다. 트리들이 독립적이라 병렬화 효과가 큼니다.
11	GridSearchCV(rf_clf, param_grid=params, cv=2, n_jobs=-1)	cv=2 로 폴드를 2개만 사용합니다. 보통 5를 쓰지만, 36개 조합 $\times$ 5폴드 = 180번 학습은 너무 오래 걸리기 때문에 시간을 절약하기 위한 타협 입니다.

⚠ 메모리 사용에 주의

n_jobs=-1 을 estimator와 GridSearchCV 양쪽에 지정하면 병렬 프로세스가 중첩되어 **메모리 사용량이 급증**할 수 있습니다. 메모리가 부족한 환경에서는 한쪽만 병렬화하거나 **n_jobs=2** 처럼 구체적인 값을 지정하는 것이 안전합니다.

4.2 탐색 결과 분석

▶ 실행 결과

```
최적 하이퍼 파라미터:
{'max_depth': 10, 'min_samples_leaf': 8, 'min_samples_split': 16, 'n_estimators': 100}
최고 예측 정확도: 0.9178
```

▶ 주요 조합별 교차 검증 정확도 (36개 중 발췌)

max_depth=6	min_samples_leaf=8	min_samples_split=8	mean=0.911725
max_depth=6	min_samples_leaf=12	min_samples_split=8	mean=0.909684
max_depth=6	min_samples_leaf=18	min_samples_split=16	mean=0.910909
max_depth=8	min_samples_leaf=8	min_samples_split=8	mean=0.915397
max_depth=8	min_samples_leaf=12	min_samples_split=16	mean=0.913221
max_depth=8	min_samples_leaf=18	min_samples_split=8	mean=0.913357
max_depth=10	min_samples_leaf=8	min_samples_split=16	mean=0.917845 ← 최적
max_depth=10	min_samples_leaf=12	min_samples_split=8	mean=0.916893
max_depth=10	min_samples_leaf=18	min_samples_split=20	mean=0.916893
max_depth=12	min_samples_leaf=8	min_samples_split=20	mean=0.916757
max_depth=12	min_samples_leaf=12	min_samples_split=8	mean=0.914853
max_depth=12	min_samples_leaf=18	min_samples_split=20	mean=0.916077

■ 결과 해석

- **max_depth=10**이 전반적으로 가장 우수합니다. 6은 너무 얇아 과소적합, 12는 약간 과적합 경향을 보입니다.
- 전체 조합의 정확도가 **0.9097~0.9178**의 좁은 범위에 몰려 있습니다. 이는 랜덤 포레스트가 하이퍼 파라미터에 덜 민감하고 안정적이라는 뜻입니다. 결정 트리가 0.844~0.855로 흔들렸던 것과 대조적입니다.
- **min_samples_split**은 값이 바뀌어도 결과가 거의 같습니다. **min_samples_leaf**가 이미 8 이상이라 분할 제약이 먼저 걸려 split 조건이 무의미해지기 때문입니다.

⚠ 결과 재현에 대하여

본 문서의 최적 조합은 **min_samples_split=16**이지만, 교재에서는 **8**로 나올 수 있습니다. **0.9178 vs 0.9169**처럼 소수점 셋째 자리 차이로 순위가 갈리기 때문이며, 사이킷런 버전이나 실행 환경에 따라 동점에 가까운 조합 중 어느 것이 선택되는지가 달라질 수 있습니다. 성능상 실질적 차이는 없습니다.

5. 최적 파라미터로 재학습

```
rf_clf1 = RandomForestClassifier(n_estimators=300, max_depth=10, min_samples_leaf=8, \
                                min_samples_split=8, random_state=0)
rf_clf1.fit(X_train, y_train)
pred = rf_clf1.predict(X_test)
print('예측 정확도: {0:.4f}'.format(accuracy_score(y_test, pred)))
```

줄	코드	상세 설명
1	n_estimators=300	중요한 변경점입니다. 튜닝 때는 100이었던 트리 개수를 300으로 3배 늘렸습니다. 탐색은 빠르게, 최종 학습은 충실하게 하는 전략입니다. 트리가 많을수록 일반적으로 성능이 조금씩 향상되며, 과적합 위험은 거의 없습니다(다만 시간은 늘어남).
1~2	max_depth=10, min_samples_leaf=8, min_samples_split=8	GridSearchCV가 찾은 최적 파라미터를 적용합니다. 줄 끝의 \ 는 줄 연결 문자입니다.
3~5	fit / predict / accuracy_score	테스트 세트에서 최종 성능을 측정합니다.

▶ 실행 결과

예측 정확도: 0.9165
(학습 13.5초)

■ 결과 해석 — 튜닝이 항상 이기지는 않는다

모델	설정	테스트 정확도
랜덤 포레스트 (기본)	제약 없음, 트리 100개	0.9223
랜덤 포레스트 (튜닝)	max_depth=10 등, 트리 300개	0.9165

흥미롭게도 튜닝한 모델이 기본 모델보다 약간 낮습니다. 이유는 다음과 같습니다.

- 랜덤 포레스트는 이미 과적합에 강하도록 설계되어 있어, 결정 트리처럼 강한 규제가 필요하지 않습니다.
- max_depth=10 , min_samples_leaf=8 은 오히려 과한 제약이 되어 개별 트리의 표현력을 떨어뜨렸습니다.
- cv=2 라는 얇은 교차 검증으로 탐색했기 때문에 추정이 불안정했을 수 있습니다.

교훈: 알고리즘마다 튜닝의 효과가 다릅니다. 결정 트리는 규제가 필수였지만, 랜덤 포레스트는 기본값이 이미 충분히 좋은 경우가 많습니다. 항상 기본 성능과 비교하는 습관이 중요합니다.

6. 피쳐 중요도 시각화

```
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

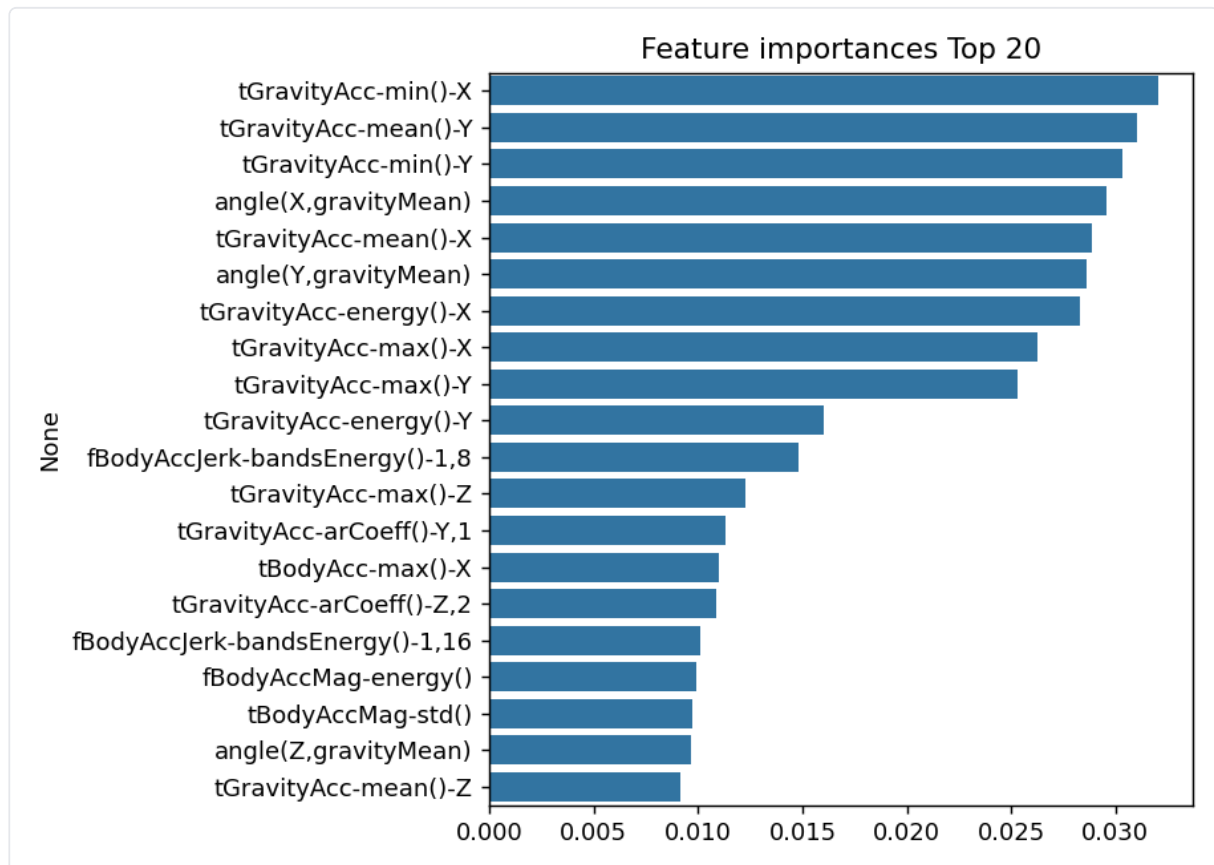
ftr_importances_values = rf_clf1.feature_importances_
ftr_importances = pd.Series(ftr_importances_values, index=X_train.columns )
ftr_top20 = ftr_importances.sort_values(ascending=False)[:20]

plt.figure(figsize=(8,6))
plt.title('Feature importances Top 20')
sns.barplot(x=ftr_top20 , y = ftr_top20.index)
fig1 = plt.gcf()
plt.show()
plt.draw()
fig1.savefig('rf_feature_importances_top20.tif', format='tif', dpi=300, bbox_inches='tight')
```

줄	코드	상세 설명
5	rf_clf1.feature_importances_	랜덤 포레스트도 결정 트리와 동일하게 피쳐 중요도를 제공합니다. 다만 300개 트리의 중요도를 평균한 값입니다.
6	pd.Series(ftr_importances_values, index=X_train.columns)	중요도에 피쳐명을 인덱스로 붙여 Series로 만듭니다.
7	sort_values(ascending=False)[:20]	내림차순 정렬 후 상위 20개 만 추출합니다.
12	fig1 = plt.gcf()	gcf() 는 Get Current Figure 의 약자로, 현재 그림 객체를 가져옵니다. plt.show() 를 호출하면 그림이 비워지기 때문에 , 저장을 위해 미리 참조를 확보 해 두는 것입니다.
14	plt.draw()	그림을 다시 그려 저장 전에 내용이 반영 되도록 합니다.
15	fig1.savefig('...tif', format='tif', dpi=300, bbox_inches='tight')	파일로 저장합니다. dpi=300 은 인쇄용 고해상도 , bbox_inches='tight' 는 여백을 최소화 합니다. tif 형식은 출판용으로 쓰입니다.

▶ 실행 결과

tGravityAcc-min()-X	0.032073
tGravityAcc-mean()-Y	0.031032
tGravityAcc-min()-Y	0.030360
angle(X,gravityMean)	0.029573
tGravityAcc-mean()-X	0.028894
angle(Y,gravityMean)	0.028642
tGravityAcc-energy()-X	0.028328
tGravityAcc-max()-X	0.026250
tGravityAcc-max()-Y	0.025300
tGravityAcc-energy()-Y	0.016015
fBodyAccJerk-bandsEnergy()-1,8	0.014834
tGravityAcc-max()-Z	0.012252
tGravityAcc-arCoeff()-Y,1	0.011294
tBodyAcc-max()-X	0.010963
tGravityAcc-arCoeff()-Z,2	0.010891
fBodyAccJerk-bandsEnergy()-1,16	0.010127
fBodyAccMag-energy()	0.009939
tBodyAccMag-std()	0.009720
angle(Z,gravityMean)	0.009624
tGravityAcc-mean()-Z	0.009114



랜덤 포레스트 피쳐 중요도 상위 20개

■ 결과 해석 — 결정 트리와의 결정적 차이

구분	결정 트리 (4.2장)	랜덤 포레스트
1위 중요도	0.2534	0.0321
상위 5개 합계	약 82%	약 15%
분포 형태	극소수에 집중	넓게 분산

결정 트리는 1위 피처가 25%를 차지했지만, 랜덤 포레스트는 **1위조차 3.2%에 불과**합니다. 이는 **피처 샘플링** 때문입니다. 각 분할에서 일부 피처만 후보에 넣으므로, 강력한 피처가 없을 때는 **차선의 피처들도 기회를 얻습니다**.

그 결과 **더 많은 피처의 정보를 골고루 활용**하게 되어 일반화 성능이 높아집니다. 이것이 랜덤 포레스트가 단일 트리보다 우수한 근본 이유 중 하나입니다.

내용 면에서는 두 모델 모두 **tGravityAcc (중력 가속도)**와 **angle(...,gravityMean) (중력과의 각도)** 계열이 상위를 차지합니다. 몸의 **자세와 방향**이 동작 인식의 핵심임을 두 모델이 일관되게 가리키고 있습니다.

7. 핵심 요약 및 머신러닝 활용 포인트

7.1 핵심 API 요약표

파라미터 / 속성	역할	튜닝 방향
<code>n_estimators</code>	결정 트리 개수 (기본 100)	늘릴수록 성능 ↑ 시간 ↑ 과적합 위험은 거의 없음
<code>max_depth</code>	트리 최대 깊이	기본 None. 과적합 시 제한
<code>min_samples_leaf</code>	리프 최소 샘플 수	과적합 시 높임
<code>min_samples_split</code>	분할 최소 샘플 수	과적합 시 높임
<code>max_features</code>	분할에 고려할 피처 수	분류 기본값 <code>'sqrt'</code> 랜덤성의 핵심 파라미터
<code>n_jobs</code>	병렬 처리 코어 수	<code>-1</code> 이면 전체 사용 메모리 부족 시 값 제한
<code>oob_score</code>	OOB 데이터로 검증	<code>True</code> 면 <code>oob_score_</code> 사용 가능
<code>feature_importances_</code>	피처별 평균 중요도	전체 트리의 평균

7.2 실습 결과 종합

모델	설정	테스트 정확도	학습 시간
결정 트리 (4.2장 최종)	<code>max_depth=8, min_samples_split=16</code>	0.8717	~5초
랜덤 포레스트 (기본)	<code>n_estimators=100, 제약 없음</code>	0.9223	5.6초
랜덤 포레스트 (튜닝)	<code>n_estimators=300, max_depth=10 등</code>	0.9165	13.5초

7.3 머신러닝 활용 포인트

- **기본값부터 시작하기** : 랜덤 포레스트는 기본 설정만으로도 강력합니다. 본 실습에서도 기본 모델이 가장 좋았습니다. **튜닝 전 기본 성능을 반드시 기록**하세요.
- **과도한 규제를 경계** : 결정 트리에 효과적이던 강한 규제가 랜덤 포레스트에는 역효과일 수 있습니다. 앙상블 자체가 이미 규제 역할을 하기 때문입니다.
- **n_estimators는 늘려도 안전** : 트리를 늘려서 과적합되는 일은 거의 없습니다. **시간이 허락하는 만큼** 늘리되, 어느 지점부터는 성능이 정체됩니다.
- **병렬 처리 활용** : `n_jobs=-1` 로 학습 시간을 크게 줄일 수 있습니다. 다만 **메모리 사용량**도 함께 늘어난다는 점을 기억하세요.
- **피쳐 중요도가 분산되는 것은 정상** : 랜덤 포레스트의 중요도는 결정 트리보다 훨씬 고르게 퍼집니다. 이는 **더 많은 정보를 활용**하고 있다는 좋은 신호입니다.
- **튜닝 시간을 절약하는 요령** : 탐색 단계에서는 `n_estimators` 를 작게, `cv` 를 2~3으로 두고, **최종 학습에서만** 트리를 늘리세요.
- **다음 단계** : 랜덤 포레스트는 **배깅** 계열의 대표 주자입니다. 다음 장의 **GBM(4.5)**부터는 **부스팅** 계열로, 순차 학습을 통해 오류를 보정하는 전혀 다른 접근을 배웁니다.

⚠ 안정화 버전 참고 (scikit-learn 1.7.x 기준)

- `n_estimators` 기본값은 **0.22부터 100**입니다(그 이전 10). 구버전 교재에서 "기본 10개"라고 설명한다면 현재와 다릅니다.
- `max_features` 기본값이 **1.1부터 'sqrt'** 로 표기가 바뀌었습니다(과거 'auto' 는 제거). 동작은 동일하게 $\sqrt{\text{전체 피쳐 수}}$ 입니다.
- `fit()` 에 2차원 `y` 를 넘기면 **DataConversionWarning**이 발생합니다. `y_train.values.ravel()` 로 1차원 변환을 권장합니다.
- `n_jobs=-1` 을 estimator와 GridSearchCV에 **중첩 지정**하면 메모리 사용량이 크게 늘 수 있습니다. 제한된 환경에서는 한 쪽만 병렬화하세요.